

FIAP 2026 – Ciência da Computação

Sistema Inteligente de Gerenciamento da Infraestrutura da Colônia

SIGC

<https://github.com/timsleo/sigic>

Atividade Integradora – Fase 4

Leonardo Tims – RM 571048

1. Organização da Infraestrutura da Colônia

O SIGIC (Sistema Inteligente de Gerenciamento da Infraestrutura da Colônia) representa a quarta fase da Missão Aurora Siger. Após a decolagem, o pouso e o gerenciamento básico da colônia, o desafio agora é representar computacionalmente a infraestrutura completa da base e otimizar sua rede energética e operacional utilizando grafos e algoritmos de redes.

A base é composta por oito módulos de infraestrutura com prioridade operacional decrescente: sistemas essenciais à vida humana recebem prioridade máxima, seguidos pelos de infraestrutura crítica e, por último, os de suporte operacional.

ID	Módulo	Prioridade	Consumo (kW)	Capacidade (kWh)	Status
0	Habitação	1 – Máxima	15,0	80,0	Ativo
1	Centro de Controle	2 – Alta	12,0	50,0	Ativo
2	Suporte Médico	3 – Alta	10,0	40,0	Ativo
3	Armazenamento Energia	4 – Média	5,0	200,0	Ativo
4	Produção de Oxigênio	5 – Média	8,0	30,0	Ativo
5	Laboratório Científico	6 – Média	9,0	45,0	Ativo
6	Agricultura	7 – Básica	7,0	60,0	Manutenção
7	Comunicação	8 – Básica	6,0	25,0	Ativo

1.1 Estruturas de Dados Utilizadas

Os dados da infraestrutura são organizados em quatro estruturas complementares, cada uma escolhida pela sua adequação ao tipo de operação realizada:

Estrutura	Uso no SIGIC	Justificativa
Tuplas	Dados fixos de cada módulo (nome, prioridade, consumo...)	Imutáveis, garantem que atributos fixos não sejam alterados acidentalmente durante a execução
Dicionários	Acesso por nome de módulo	Busca $O(1)$ por chave, ideal para consultas rápidas durante operação da colônia
Listas	Módulos ordenados por ID; lista de adjacência (vizinhos)	Iteração sequencial eficiente; armazena apenas conexões existentes. $O(V + E)$
Matrizes	Matriz de adjacência $V \times V$ com pesos (distâncias)	Verificação de aresta em $O(1)$; adequada para grafos relativamente densos como a rede da colônia

2. Representação da Rede Utilizando Grafos

A infraestrutura da colônia é representada como um grafo não direcionado ponderado. Os módulos são os vértices e as conexões físicas entre eles são as arestas. Os pesos representam a distância em metros entre os módulos, utilizados para calcular rotas de distribuição de energia e comunicação.

2.1 Arestas e Pesos da Rede

Módulo A	Módulo B	Distância (m)
Habitação	Centro de Controle	50
Habitação	Suporte Médico	80
Habitação	Produção de Oxigênio	60
Centro de Controle	Armazenamento Energia	40
Centro de Controle	Laboratório Científico	70
Centro de Controle	Comunicação	55
Suporte Médico	Armazenamento Energia	45
Armazenamento Energia	Produção de Oxigênio	35
Armazenamento Energia	Agricultura	90
Armazenamento Energia	Laboratório Científico	65
Produção de Oxigênio	Agricultura	75
Laboratório Científico	Comunicação	50
Agricultura	Comunicação	85

2.2 Representações em Memória

O SIGIC implementa as duas representações clássicas de grafos, utilizando cada uma onde é mais eficiente:

Critério	Matriz de Adjacência	Lista de Adjacência
Espaço	$O(V^2)$	$O(V + E)$
Verificar aresta (u,v)	$O(1)$ — consulta <code>mat[u][v]</code>	$O(\text{grau}(u))$ — percorre lista
Listar vizinhos de u	$O(V)$ — varre linha inteira	$O(\text{grau}(u))$ — percorre vizinhos
Uso no SIGIC	BFS e Dijkstra (varredura da linha)	DFS e detecção de componentes

```
# Matriz de adjacência: mat[i][j] = distância; 0 = sem aresta
def criar_matriz_adjacencia():
    mat = [[0] * N for _ in range(N)]
    for a, b, w in ARESTAS:
        i = modulos_dict[a]['id']
        j = modulos_dict[b]['id']
        mat[i][j] = w
        mat[j][i] = w # grafo não direcionado
    return mat

# Lista de adjacência: adj[u] = [(v, peso), ...]
def criar_lista_adjacencia():
    adj = {i: [] for i in range(N)}
    for a, b, w in ARESTAS:
        i = modulos_dict[a]['id']
        j = modulos_dict[b]['id']
        adj[i].append((j, w))
        adj[j].append((i, w))
    return adj
```

3. Implementação dos Algoritmos de Rede

3.1 BFS — Busca em Largura

A BFS percorre a rede por camadas, visitando primeiro os módulos mais próximos da origem em número de saltos. Utiliza uma fila (deque) para controlar a ordem de processamento e um vetor `parent[]` para reconstrução do caminho.

```
from collections import deque def bfs(origem_id): visited = [False] * N parent = [-1] * N ordem = [] fila = deque() visited[origem_id] = True fila.append(origem_id) while fila: u = fila.popleft() # FIFO ordem.append(u) for v in range(N): # varre linha da matriz if MATRIZ[u][v] > 0 and not visited[v]: visited[v] = True parent[v] = u fila.append(v) return ordem, parent
```

3.2 DFS — Busca em Profundidade

A DFS avança o máximo possível em uma direção antes de retornar (backtracking). Implementada de forma recursiva com lista de adjacência — percorre apenas os vizinhos existentes de cada vértice.

```
def dfs(u, visited, ordem, adj): visited[u] = True ordem.append(u) for v, _ in adj[u]: # percorre vizinhos diretamente if not visited[v]: dfs(v, visited, ordem, adj) # chamada recursiva
```

3.3 Dijkstra — Caminho Mínimo Ponderado

O Dijkstra encontra a menor distância total (em metros) entre dois módulos. Implementado na versão didática $O(V^2)$: seleção do menor `dist[]` por varredura e relaxamento de arestas. O vetor `parent[]` é reutilizado para reconstrução do caminho — mesma técnica da BFS.

```
INF = float('inf') def dijkstra(origem_id): dist = [INF] * N parent = [-1] * N usado = [False] * N dist[origem_id] = 0 for _ in range(N): # Seleciona vértice não finalizado com menor dist[] u = -1 for i in range(N): if not usado[i] and dist[i] < INF: if u == -1 or dist[i] < dist[u]: u = i if u == -1: break usado[u] = True # Relaxamento das arestas saindo de u for v in range(N): w = MATRIZ[u][v] if w > 0 and not usado[v]: if dist[u] + w < dist[v]: dist[v] = dist[u] + w parent[v] = u return dist, parent
```

3.4 Exemplo de Saída — Dijkstra (Habitação como origem)

```
Distâncias a partir de 'Habitação': Habitação : 0 m Centro de Controle : 50 m Suporte Médico : 80 m Armazenamento Energia : 90 m Produção de Oxigênio : 60 m Laboratório Científico : 120 m Agricultura : 135 m Comunicação : 105 m Melhor rota até 'Armazenamento Energia': Habitação -> Centro de Controle -> Armazenamento Energia Distância total: 90 m
```

3.5 Comparativo dos Algoritmos

Algoritmo	Estrutura usada	Complexidade	Uso no SIGIC
BFS	Fila (deque) + matriz	$O(V^2)$	Menor número de saltos entre módulos
DFS	Recursão + lista adj.	$O(V + E)$	Exploração da rede e detecção de componentes
Dijkstra	Matriz + varredura	$O(V^2)$	Menor distância total em metros entre módulos

4. Modelagem Matemática e Otimização

Para apoiar as decisões operacionais do SIGIC, foi modelado matematicamente o consumo energético da colônia em função do número de módulos ativos. A função escolhida é quadrática, pois cada módulo adicional gera dependências com os demais, aumentando o consumo de forma acelerada.

4.1 Função Matemática

$$E(n) = a \cdot n^2 + b \cdot n + c$$

Parâmetro	Símbolo	Valor	Significado
Fator de crescimento	a	0,3 kW/módulo ²	Interdependências energéticas entre módulos
Consumo base por módulo	b	9,0 kW	Consumo médio dos módulos ativos da colônia
Consumo fixo	c	20 kW	Energia mínima para manter a base operacional
Módulos ativos	n	variável	Número de módulos em operação simultânea

4.2 Cálculo Diferencial com SymPy

```
import sympy as sp
n = sp.Symbol('n', positive=True)
E = sp.Rational(3,10)*n**2 + 9*n + 20
dE_dn = sp.diff(E, n) # primeira derivada
d2E_dn2 = sp.diff(E, n, 2) # segunda derivada
# Resultados:
# E(n) = 3*n**2/10 + 9*n + 20
# dE/dn = 3*n/5 + 9
# d2E/dn2 = 3/5 (> 0: concavidade para cima)
```

4.3 Análise Qualitativa

A segunda derivada $d^2E/dn^2 = 3/5 > 0$ confirma que a função é côncava para cima: o consumo cresce de forma acelerada com o número de módulos ativos. A derivada $dE/dn = 3n/5 + 9$ é sempre positiva para $n > 0$, indicando que o consumo nunca decresce ao adicionar módulos. O custo marginal de energia por módulo adicional aumenta continuamente.

4.4 Otimização por Gradiente Descendente

```
f_num = sp.lambdify(n, E, 'numpy')
n_val = 8.0 # ponto inicial: todos os módulos ativos
alpha = 0.01 # taxa de aprendizagem
eps = 1e-5 # tolerância
for _ in range(200):
    # Derivada numérica por diferença centrada
    grad = (f_num(n_val + eps) - f_num(n_val - eps)) / (2 * eps)
    n_val = n_val - alpha * grad
n_val = max(1.0, n_val) # restrição: mínimo 1 módulo
```

4.5 Relação com as Decisões de Engenharia

Decisão Operacional	Como E(n) Auxilia
Expansão da colônia	Calcula $E(n+1) - E(n) = dE/dn$ antes de ativar um novo módulo, verificando se a capacidade de armazenamento suporta o acréscimo.
Desligamento de emergência	A derivada indica quais módulos, ao serem desligados, geram maior redução de consumo, priorizando os de menor prioridade.
Planejamento de capacidade	A função quadrática projeta o consumo futuro para N módulos, dimensionando painéis solares e armazenamento com antecedência.

5. Contextualização Histórica da Computação

O SIGIC, como sistema embarcado de alta confiabilidade para um ambiente hostil, é herdeiro direto de décadas de evolução computacional. Desde as primeiras máquinas de propósito geral até os sistemas de navegação espacial modernos.

5.1 Dos Computadores de Propósito Geral aos Sistemas Embarcados

O ENIAC (1945), primeiro computador eletrônico de propósito geral, ocupava uma sala inteira e foi criado para cálculos balísticos. Com a invenção do transistor (1947) e dos circuitos integrados (1960), os computadores foram miniaturizados progressivamente. O Apollo Guidance Computer (1969), que guiou o homem à Lua operando com apenas 4 KB de memória, é o predecessor direto de sistemas como o SIGIC: pequeno, confiável, com algoritmos simples e decisões precisas em tempo real.

Rovers modernos em Marte, como o Perseverance, utilizam computadores tolerantes à radiação com processadores equivalentes aos de 1998, priorizando confiabilidade sobre velocidade. Esse princípio guia diretamente as escolhas de projeto do SIGIC.

5.2 Limitações de Hardware em Missões a Marte

Limitação	Causa	Impacto no SIGIC
Memória limitada	Custo, peso e blindagem contra radiação cósmica	Uso de listas e matrizes simples em vez de estruturas complexas como árvores B
Processamento restrito	Chips rad-hard em baixas frequências	Dijkstra $O(V^2)$ sem heap: comportamento previsível, sem estruturas auxiliares
Energia limitada	Painéis solares recebem 43% da irradiação terrestre	Algoritmos enxutos; sem processamento em nuvem
Tolerância a falhas	Comunicação com atraso de 3 a 22 minutos	Lógica explícita com IF/ELIF/ELSE; sem modelos probabilísticos
Radiação cósmica	Marte sem magnetosfera protetora significativa	Estruturas simples são mais resilientes a erros de bit isolados

5.3 Influência nas Escolhas de Projeto

Cada escolha de algoritmo e estrutura de dados no SIGIC reflete as restrições de hardware de uma missão em Marte. O Dijkstra didático $O(V^2)$ sem fila de prioridade mantém o foco na lógica de relaxamento com comportamento completamente previsível. As tuplas garantem imutabilidade dos dados críticos dos módulos, reduzindo o risco de corrupção em memória sujeita a erros de bit. Assim como os engenheiros da Apollo comprometeram desempenho por confiabilidade, o SIGIC prioriza clareza e previsibilidade sobre sofisticação algorítmica.

6. Sustentabilidade e Governança da Infraestrutura

A operação da base Aurora Siger deve ir além da viabilidade técnica. As decisões computacionais do SIGIC carregam implicações ambientais, sociais e de governança que precisam ser estruturadas desde o primeiro dia da colônia.

6.1 E — Ambiental: Uso Sustentável de Energia

A modelagem matemática $E(n) = 0,3n^2 + 9n + 20$ revela que o consumo cresce de forma acelerada com novos módulos. O SIGIC utiliza essa função para orientar decisões de ativação: antes de ligar um novo módulo, calcula o custo marginal dE/dn e verifica se a geração solar corrente suporta o acréscimo. Módulos em manutenção (como a Agricultura) permanecem desligados até que a margem energética permita reativação segura.

6.2 S — Social: Expansão Organizada e Priorização de Sistemas Críticos

O algoritmo de Dijkstra orienta a expansão física da base: novas conexões entre módulos devem minimizar a distância total da rede, reduzindo perdas de transmissão. Em situações de escassez energética, o SIGIC desliga módulos na ordem inversa de prioridade (Comunicação e Agricultura primeiro) enquanto Habitação, Suporte Médico e Produção de Oxigênio permanecem sempre operacionais.

6.3 G — Governança: Ética e Responsabilidade Tecnológica

Princípio	Aplicação no SIGIC
Auditabilidade do código	Todo o código-fonte é aberto, comentado e revisável pelos membros técnicos da colônia. Cada decisão algorítmica tem justificativa explícita.
Supervisão humana	O SIGIC exibe rotas e análises, mas não executa ações autônomas de desligamento. Toda intervenção crítica requer confirmação humana.
Transparência dos dados	Todos os atributos dos módulos (consumo, status, prioridade) são exibidos integralmente na opção de consulta, sem filtros ocultos.
Redução de desperdícios	A análise energética calcula continuamente a eficiência da rede, identificando módulos com maior consumo relativo e sugerindo ajustes.